

Q.5) Convert BST $\rightarrow$ min Heap
(complete)

approach: 2 conditions check

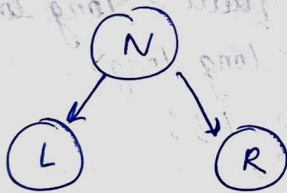
- 1) Min Order property
- 2) $\forall L < \forall R$

NOTE:

if it's already CBT $\rightarrow$ it's struct is going to be same.

$\hookrightarrow$ so change only values of node.

BST $\rightarrow$ inorder $\rightarrow$ condition
(sorted) $(N < L < R)$



NLR (preorder)

insert preorder wise in tree.

```
fillPreOrder (root, inorder, index) {
    root  $\rightarrow$  data = inorder[index++];
    fillPreOrder (root  $\rightarrow$  left, inorder, index);
    fillPreOrder (root  $\rightarrow$  right, inorder, index);
}
```

Q.76

Q.1) K^{th} largest sum Subarray

i.p = arr[] = {1, 2, 6, 4, 3}

output = from all possible sum of subarrays return K^{th} largest value.

approach 1: Brute force T.C ($n^2 \log n$)
S.C (n^2)

```
for (i=0 to n) {
    for (j=i to n) {
        sum += arr[i];
    }
    sumpush_back(sum);
}
```

sort(sum);

return sum[n-k];

}

approach 2: Min Heap
int getKth(arr, k) {
 priority-queue <int, vector<int>,
 greater<int>> mini;

int n = arr.size();

```
for (int i=0; i<n; i++) {
    int sum=0;
```

```
    for (int j=i; j<n; j++) {
        sum += arr[j];
```

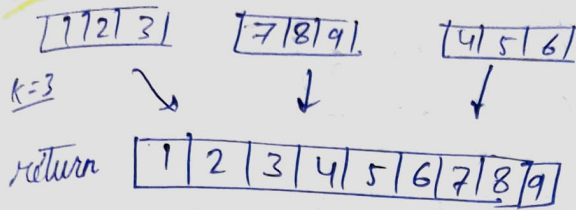
```
        if (mini.size() < k) {
```

```
            mini.pop();
```

```
            mini.push(sum);
```

```
        }
    }
    return mini.top();
}
```

Q.2) Merge k sorted arrays.



approach:

- ① Create min heap having size 'k'.
- ② push_back (mini.top()) in vector.

→ while mini.size() > 1.

NOTE: we also have to track we are pushing from which array to inc the index.

code

Class Node {

public:

int data;

int i;

int j;

};

constructor.

node (int data, int row, int col) {

this->data = data;

i = row;

j = col;

}

class compare {

public:

bool operator() (Node* a, Node* b) {

return a->data > b->data;

}

};

vector<int> mergeKSortedArrays(
vector<vector<int>> &kArrays,
int k) {

priority_queue<node*, vector<node*>, compare> minHeap;

// step 1: store arrays k first element insert in minHeap.

for (int i=0; i<k; i++) {
node* tmp = new node (kArrays[i][0],
i, 0);
minHeap.push(tmp);
}

vector<int> ans;

// step 2:

while (minHeap.size() > 0) {

ans.push_back(minHeap.top().data);

minHeap.pop();

int i = minHeap.top().i;

node* tmp = minHeap.top();

ans.push_back(tmp->data);

minHeap.pop();

int i = tmp->i;

int j = tmp->j;

if (j+1 < kArrays[i].size()) {

node* next = new node (

kArrays[i][j+1], i,
j+1);

minHeap.push(next);

}

return ans;

Q.3) Merge K sorted linked list.

approach 1:

- ① Copy all LL
- ② Sort last LL (final).

approach 2:

min heap $\rightarrow$ first elements of K linked list.

now only focus on decreasing range

$[1, 5]$
 $\rightarrow \leftarrow$
 $[10, 2] \rightarrow [2, 10]$
 $[0(K)]$ S.L.
 $T.C \rightarrow [0(nk^2)]$

always keep inc minimum element and store range if its minimum.

lc77

Q.1) ^{***} Smallest Range of K-lists. (sorted)

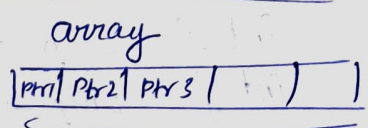
$\hookrightarrow$ return smallest range $[x, y]$ such that atleast one element from each list lies in $[x, y]$.

approach 1: find all ranges & return smallest one.

e.g. $K=3$
 $\{1, 10, 11\}$
 $\{2, 3, 20\}$
 $\{5, 6, 12\}$

$[0(n^2k^2)]$

#2)



K size

$ptr1 \rightarrow \{1, 10, 11\}$
 $ptr2 \rightarrow \{2, 3, 20\}$
 $ptr3 \rightarrow \{5, 6, 12\}$

stored values are

1	2	5
---	---	---

$\hookrightarrow$ range = $[min, max] = [1, 5]$

#3) Heaps algo

- ① create min heap of K size and put all starting elements.
- ② Track min & max
- ③ while (!pq.empty()) {
 min = pq.top();
 pq.pop();
 if (max-min < ansEnd - ansStart)
 {
 update ans;
 }
 update max;
 }

code

```

class node {
public:
    int data;
    int row;
    int col;

    node (int d, int r, int c) {
        data = d;
        row = r;
        col = c;
    }
}
    
```

```

class compare {
public:
    bool operator () (node a,
                      node b) {
        return a->data > b->data;
    }
}

```

```

int range (vector < vector < int >>
           &a, int k, int n) {

```

```

    int mini = INT_MAX;
    int maxi = INT_MIN;

```

```

    priority_queue < node*, vector < node* >,
    compare > minHeap;

```

```

    for (int i=0; i<k; i++) {
        int element = a[i][0];
        maxi = max (maxi, element);
        minHeap.push_back (element);
        mini = min (mini, element);
    }

```

```

    int s = mini, e = maxi;
    while (!minHeap.empty()) {

```

```

        node* temp = minHeap.top();
        minHeap.pop();

```

```

        mini = temp->data;

```

```

        // update range

```

```

        if (maxi - mini < e - s) {
            s = mini;
            e = maxi;
        }

```

```

    if (temp->col < n) {
        maxi = max (maxi,
                    a[temp->row][temp->col+1]);
        minHeap.push (new node (
            a[temp->row][temp->col+1],
            temp->row, temp->col+1));
    } else {
        break;
    }
}
return (e - s + 1);
}

```

Q.2 *** (not sorted)
Median in a stream.

2, 3, 4 → 3

2, 3, 4, 5 → (3+4)/2 → 3

approach 1: ① Sort (Insertion)
because stream is running.

② return arr[n/2] if odd.
else

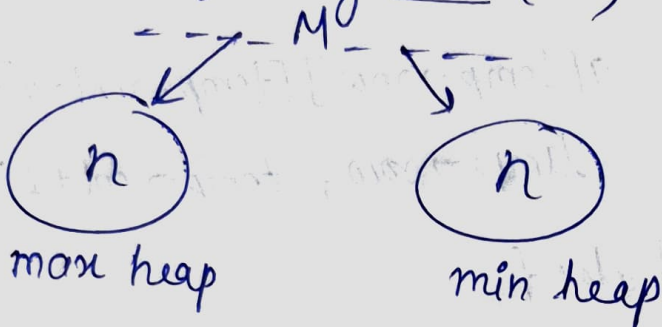
return $\frac{\text{arr}[n/2] + \text{arr}[n/2 - 1]}{2}$

$O(n^2)$

approach 2

separate array by 2 heaps.
(divide array in two parts).

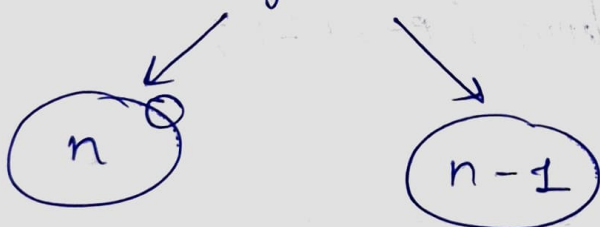
case 1 if array even ($2n$)



$$\text{median} = \frac{\text{max.top() + min.top()}}{2}$$

case 2

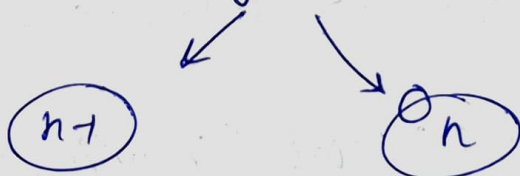
array odd ($2n+1$)



return max.top();

case 3

array odd ($2n-1$)



return min.top();

HASH MAPS

→ A type of data structure which can be used to map no. of occurrence & calculate frequency. (e.g. occurrence of alphabets in string).

→ we use 'map' <_, _> data structure.

→ Insertion
Deletion
Search } T.C = $O(1)$

#include <map>

#include <unordered_map>

unordered_map < string, int > um;

um.at("unknownKey");

↳ gives error (not in range)

um["unknownKey"];

↳ return 0 (because it makes a new key).

↳ now .at("un") ⇒ also gives '0'.

um.count("bro") → 0 (absent)
→ 1 (present).

um.erase("love")

↳ erases key "love".

Iterator in map (unordered).

unordered_map <_, _> :: iterator
it = um.begin();

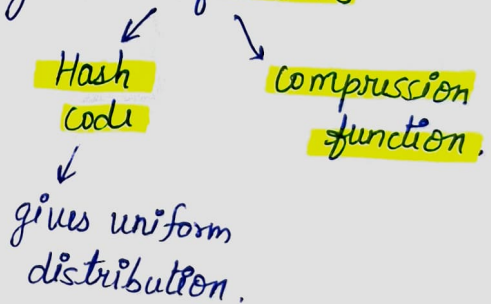
```
while (it != m.end()) {  
    cout << it->first << " " <<  
        it->second << endl;  
    it++;  
}
```

Bucket Array.

map < key, value >

↳ mapping of strings to int value (indices) of arrays.

using Hash functions.



Collision → two different strings leads to have same key value in map.

Collision Handling

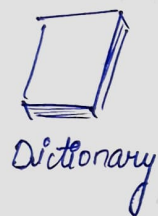
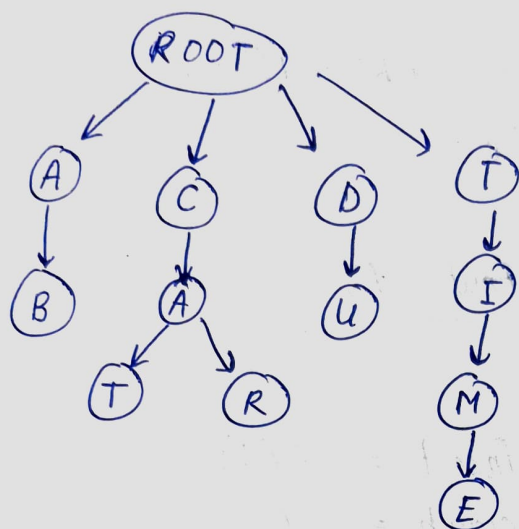
- ↳ open hashing (use same place)
- ↳ closed Addressing

Complexity Analysis

$$T.C = O(1)$$

$$S.C = O(k)$$

↳ length of words.

TRIE

Dictionary

-
- 1) add word
 - 2) search
 - 3) remove word.

Implementation:

```

class TrieNode {
public:
    char data;
    TrieNode* children[26];
    bool isTerminal;

```

```

TrieNode(char data) {
    this->data = data;
    for (int i=0; i<26; i++) {
        children[i] = 0;
    }
    isTerminal = false;
}

```

```

class Trie {
    TrieNode* root;

```

```

    void insertUtil(TrieNode* root,
        string word) {

```

// base case

```

    if (word.length() == 0) {
        root->isTerminal = true;
    }
    return;

```

```

    int index = word[0] - 'A';
    TrieNode* child;

```

// present

```

    if (root->children[index] != NULL) {
        child = root->children[index];
    } else {
        // absent
        child = new TrieNode(word[0]);
        root->children[index] = child;
    }

```

// Recursion.

```

    insertUtil(root, word.substr(1));
}

```

```

void insertWord(string word) {
    insertUtil(root, word);
}

```

```

Trie() {

```

```

    root = new TrieNode('0');
}

```

```
int main() {
```

```
    Trie *t = new TrieNode();
```

```
    t → insertWord("abcd");
```

```
    } return 0;
```

inside class Trie...

```
bool searchUtil(TrieNode* root,  
string word) {
```

```
    // base;
```

```
    if (word.length() == 0) {  
        return root → isTerminal;  
    }
```

```
    index = word[0] - 'A'
```

```
    TrieNode * child;
```

```
    // present
```

```
    if (root → children[index] != NULL) {  
        child = root → children[index];
```

```
    } else { // absent  
        return false;
```

```
    }
```

```
    // Recursion
```

```
    return searchUtil(child, word.  
        substr(1));
```

```
}
```

```
bool search(string word) {  
    return searchUtil(root, word);  
}
```

```
int main() {
```

```
    t → searchWord("abcd");
```

```
    return 0;
```

```
}
```

```
    ↪ output = 1;
```

T.C

insert = $O(1)$

search = $O(1)$

deletion = $O(1)$ ↪ length of word

Remove Word:

↪ Traverse till terminal node using search function.

↪ isTerminal = false;

NOTE: Hashmap uses more space in storing each individual word but in Trie, it optimally uses space for similar words.

+

Search fⁿ also fast in Trie.